



Musubi

La memoria que *ata* tu trabajo a través del tiempo y de las máquinas.

model-free

Go + SQLite

MCP • stdio + HTTP

v0.78.0-dev

01 • EL CONCEPTO

Un cerebro que no piensa por vos — recuerda por vos.

Musubi (結び, “atar / anudar”) es un servidor de memoria para agentes de IA.

Le da al agente lo único que un modelo no tiene solo: **continuidad**. Cada decisión, gotcha, hecho de código y commit queda anudado a un hilo que sobrevive al fin de la sesión y te sigue de una máquina a otra.

Musubi es **model-free** a propósito: no juzga con un modelo propio. El **agente** —que ya es un modelo— aporta el juicio; Musubi aporta la capa determinista donde lo determinista gana: guardar, deduplicar, olvidar, *redactar secretos* y buscar.

Esa división es la tesis entera. El recall ya era automático. Lo nuevo de esta etapa: **la captura también**. Aprende mientras trabajás, sin que le pidas nada.

Memoria y orquestación, sobre el mismo motor.

PILAR I

Memoria

Recordar y recuperar el conocimiento del proyecto.

- Recall automático por turno (semántico + keyword)
- Captura automática C1–C4
- Scope local / shared · dedup · decay · bi-temporal
- Redacción de secretos en el borde compartido

PILAR II

Orquestación / SDD

Cómo se hace el trabajo, con rigor.

- Skills cognitivas (analyze, plan, audit, orchestrate...)
- Flujo SDD: proposal → spec → design → tasks → verify
- Contract-net · claims con lease/TTL
- Telemetría de errores → resolución

De la interfaz al cerebro central.

Interfaz

lo que toca el agente

Tools MCP (`musubi_recall` `save_*` `search_*` `sdd`) + CLI (`setup` `provision` `maintain`).

Automatización

hooks de Claude Code

`SessionStart` → `detect` `UserPromptSubmit` → `turn`
`PreToolUse(Read)` → `precheck` `Stop` → `capture`

Motor

determinista, model-free

`DbEngine` · `SQLite` + `FTS5` · embeddings estáticos (POTION) · dedup trigram · decay Ebbinghaus · redacción RE2 + entropía.

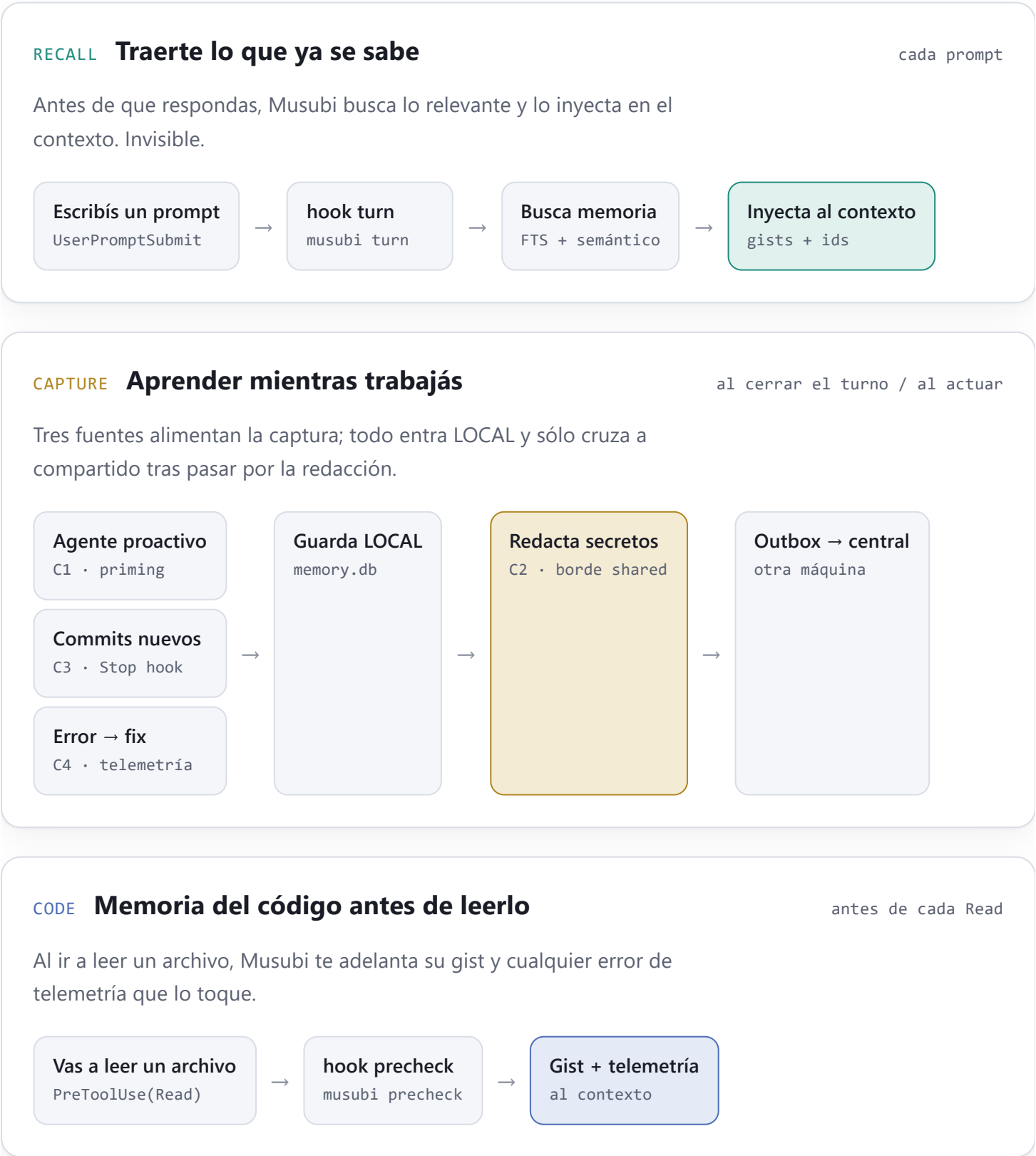
Cerebro

local ↔ central

`.musubi/memory.db` por proyecto → `outbox sync` → central
`100.79.126.62:7717` sobre Tailscale.

Todo lo que pasa sin que lo pidas.

Cada loop lo dispara un hook. El agente nunca “decide” invocar memoria: sucede alrededor de él.



La memoria te sigue de la PC a la laptop.

Captura LOCAL + scope shared + outbox sync = lo que aprendés en una máquina aparece en la otra, offline-first, sobre la red privada.



Lo automatizado, ahora mismo.

AUTOMATISMO	GATILLO	ESTADO
Recall por turno	UserPromptSubmit → turn	● vivo
Gist de código	PreToolUse(Read) → precheck	● vivo
Priming de arranque	SessionStart → detect	● vivo
Captura de commits · C3	Stop → capture	● vivo · probado
Captura proactiva · C1	priming del agente	● vivo
Redacción de secretos · C2	borde scope→shared	● en el motor
Captura error→fix · C4	musubi_resolve_telemetry	● listo · reabrir proyecto

Los **hooks** ya corren el binario nuevo (fresco cada invocación). Las **tools MCP** las sirve el daemon: para C4 y cualquier tool nueva, reabrí el proyecto una vez para reiniciar el MCP server.

Dónde seguir.

P3 · P4 **Provisioning completo**

P1/P1.5/P2 dejan la máquina conectada y el proyecto seteado. Faltan **P3** (dev tools: node/go/git) y **P4** (clonar repos + git/SSH). El one-liner “instalo Musubi y la máquina queda 100% lista”.

Equipo **Multi-usuario real**

Hoy “todo shared” asume un dev. Para el equipo: identidad por dev + project_id, qué comparte cada quién, y cómo se ve la memoria de otro sin ruido.

Hueco **Redacción en el ingest central**

C2 redacta en el borde local→shared. Un write directo al central con scope=local no pasa por ahí. Guarda de ingest server-side, fail-closed, como defensa en profundidad.

Salencia **Menos ruido, más señal**

La captura proactiva depende del juicio del agente. Medir qué se captura de más / de menos, y afinar decay + dedup como proxy de “esto valía”.

Release **Cortar versión**

Todo C1–C4 + P2 está en main pero el binario instalado es 0.78.0-dev. Falta tag/release y propagar a los 6 repos.